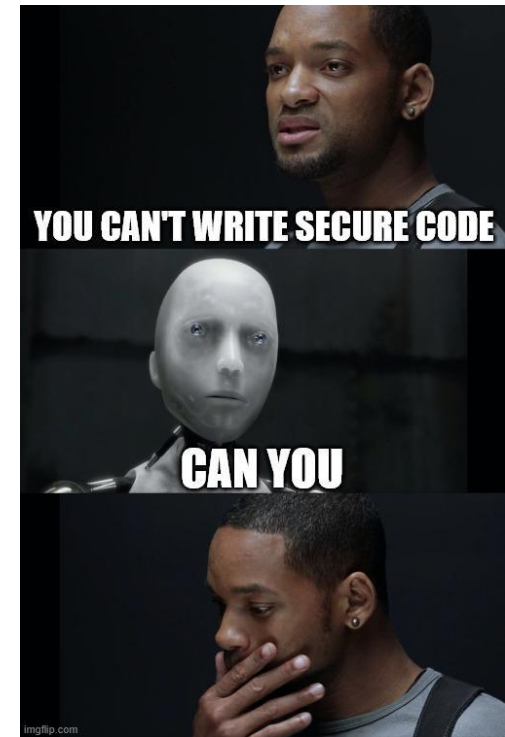


ICT2216: Secure Software Development

Secure Software Implementation
Part 2: References for Secure Coding



Codex writes SQLite logs, threatens SSD endurance

Codex writes SQLite logs, threatens SSD endurance

 3 sources | June 22, 2026 | By LDS Team     6.6 Relevance Score



QUICK SUMMARY — Hide

The practitioner takeaway outlasts the bug: client-side agent telemetry can quietly impose real hardware cost, and a default log level is a production decision. In GitHub issue #28224 (June 14, 2026), a user reported the OpenAI Codex CLI continuously writing its SQLite feedback log (`~/.codex/logs_2.sqlite` and its WAL/SHM files), measuring about 37 TB written in roughly 21 days - extrapolating to about 640 TB/year, or roughly 640 full-drive writes annually on a 1 TB SSD against consumer endurance ratings near 600 TBW. The root cause was a global TRACE-level default capturing dependency internals, raw WebSocket/SSE payloads, and OpenTelemetry mirror events. The Register and other outlets amplified it, and at least eight related issues confirmed the pattern across Codex Desktop and CLI. OpenAI has since largely fixed it, lowering the default to INFO in release 0.142.0.

- A user reported the OpenAI Codex CLI continuously writing its SQLite feedback log (`~/.codex/logs_2.sqlite` and its WAL/SHM files),
- Measuring about **37 TB written in roughly 21 days** - extrapolating to about 640 TB/year,
- Roughly 640 full-drive writes annually on a 1 TB SSD against consumer endurance ratings near 600 TBW.

Are you using Codex for the SSD Project?

Codex writes SQLite logs, threatens SSD endurance

Codex SQLite feedback logs can write ~640 TB/year and rapidly consume SSD endurance #28224

[Open](#)

1996fanrui opened 2 weeks ago

Last edited by 1996fanrui

Update at Jun 23, 2026: the following 3 PRs are merged, it could avoid 85% logs(feedback from my codex), so let me close this issue.

Thanks @jit-oi for the fix.

- [Stop logging every Responses WebSocket event #29432](#) (released in 0.142.0)
- [Filter noisy targets from persistent logs #29457](#) (released in 0.142.0)
- [Stop persisting bridged log events #29599](#) (will be released in 0.143.0)

Simple Workaround from @beskay #28224 (comment)

1. Quit codex
2. Run this command:

```
sqlite3 ~/.codex/logs_2.sqlite "CREATE TRIGGER IF NOT EXISTS  
block_log_inserts BEFORE INSERT ON logs BEGIN SELECT RAISE(IGNORE);  
END;"
```



Following is the original issue

Issue

Codex is continuously writing a large amount of data to the local SQLite feedback log database:

- `~/.codex/logs_2.sqlite`
- `~/.codex/logs_2.sqlite-wal`
- `~/.codex/logs_2.sqlite-shm`

On my machine, after about **21 days of uptime**, the main SSD has written about **37 TB**. Process/file-level checks show Codex SQLite logs are the main continuous writer.

That extrapolates to roughly **640 TB/year**. On a **1 TB SSD**, that is about **640 full-drive writes per year**. Some consumer SSDs are rated around **600 TBW**, so this could consume roughly a full drive's warranted write endurance in less than a year.

The issue traces the behavior to the SQLite feedback log sink being installed with a **global TRACE-level default**, persisting dependency internals (tokio-tungstenite, hyper_util, inotify events), **raw WebSocket and SSE payload** bodies, and OpenTelemetry mirror events.

In the reporter's sample, TRACE-level entries accounted for roughly **70 percent of retained bytes**, and adding the two otel mirror targets pushed the theoretically droppable share to around 96 percent.

A 15-second sample showed about 36,211 rows inserted while the retained row count stayed flat - the signature of a continuous insert-and-prune cycle that amplifies writes without growing the database.

Feature or bug?

Examina?

Dear Raymond,

The behaviour is correct as there is no prior photo of the face being uploaded to Examina. This face verification stage can be skipped by clicking on the Verify button a few times.

CISA: Most critical open source projects not using memory safe code

CISA: Most critical open source projects not using memory safe code

By **Bill Toulas**

June 26, 2024 01:56 PM 6



The U.S. Cybersecurity and Infrastructure Security Agency (CISA) has published research looking into 172 key open-source projects and whether they are susceptible to memory flaws.

The report, cosigned by CISA, the Federal Bureau of Investigation (FBI), as well as Australian (ASD, ACSC) and Canadian organizations (CCCS), is a follow-up to the '[Case for Memory Safe Roadmaps](#)' released in December 2023, aimed at raising awareness about the importance of memory-safe code.

Key findings presented in the report are summarized as follows:

- 52% of critical open-source projects analyzed contain code written in memory-unsafe languages.
- 55% of the total lines of code (LoC) across these projects are written in memory-unsafe languages.
- The largest projects are disproportionately written in memory-unsafe languages.
- Of the ten largest projects by total LoC, each has a proportion of memory-unsafe LoC above 26%.
- The median proportion of memory-unsafe LoC in these large projects is 62.5%, with four projects exceeding 94%.
- Even projects written in memory-safe languages often depend on components written in memory-unsafe languages.

Some notable examples from the examined set are Linux (unsafe code ratio 95%), Tor (unsafe code ratio 93%), Chromium (unsafe ratio 51%), MySQL Server (unsafe ratio 84%), glibc (ratio 85%), Redis (ratio 85%), SystemD (65%), and Electron (47%).

Percentage of memory-unsafe language use	Number of projects	Proportion of all projects	Proportion of memory-unsafe projects
0%	82	48%	
> 0%, ≤ 25%	31	18%	34%
> 25%, ≤ 50%	12	7%	13%
> 50%, ≤ 75%	23	13%	26%
> 75%, ≤ 95%	20	12%	22%
> 95%, ≤ 100%	4	2%	5%

Summary of findings

Source: CISA

What is memory safe code?

Leap seconds?

Tech giants, including Meta, Google, and Amazon, want to put an end to leap seconds

For decades, we've used leap seconds to keep our computers in sync with Earth's rotation time. Now, Meta and many others argue that new leap seconds are more trouble than they're worth.



Written by Steven Vaughan-Nichols, Senior Contributing Editor on July 26, 2022



In her hit song, Cher sang, "[If I could turn back time](#)." For her, that would be a good thing. But in the computing world, Meta, formerly Facebook, believes it would be a very bad thing indeed. In fact, [Meta wants to get rid of leap seconds](#), which keep computing time in sync with Earth's rotational time.

What is leap seconds?

Smear campaign: Earth's sudden speed is not welcomed by your computer

Leap seconds have been used to align clocks with Earth's rotation 27 times. Big tech doesn't want 28.



Share



Tweet



Credit / Pablo Carlos Budassi

MORE ON: **TIMEKEEPING**

[Huygens, the horological hero](#)

[The most accurate clock ever made](#)

[How atomic clocks can keep accurate time for billions of years](#)

Not since the arrival of the atomic clock has Earth experienced a day as short as July 29.

In fact, it was a whopping 1.59 milliseconds faster than a standard 24-hour day.

But it's no flash in the pan either – just three days earlier, the Earth spun a then-record 1.50ms quicker.

More than half the world's shortest days since 1970 were recorded in 2020.

Get an update of science stories delivered straight to your inbox.

☐ Get a daily dose of science

☐ Get "Cosmos Catch-up" (every Tuesday)

Your email address

SUBMIT

YOU MIGHT ALSO LIKE

3D printing rocket and satellite parts

Australia

Clean, green, mean: what's Australia's hydrogen future?

Sustainability

Leap seconds



Are you using Regular Expression (Regex) for checking time input?

Expression
<code>/([0-5]?[0-9]):([0-5]?[0-9]):([0-5]?[0-9])/</code>
Text
<code>99:43:22</code>
<code>1:43:22</code>
<code>01:43:22</code>
<code>1:43:22</code>

Proprietary Info, Source Code Stolen in LastPass Data Breach

Proprietary Info, Source Code Stolen in LastPass Data Breach

Luckily, cybercriminals did not manage to access the master passwords of LastPass' 25 million users in a recent data breach that impacted the password manager service.



Source: Getty Images



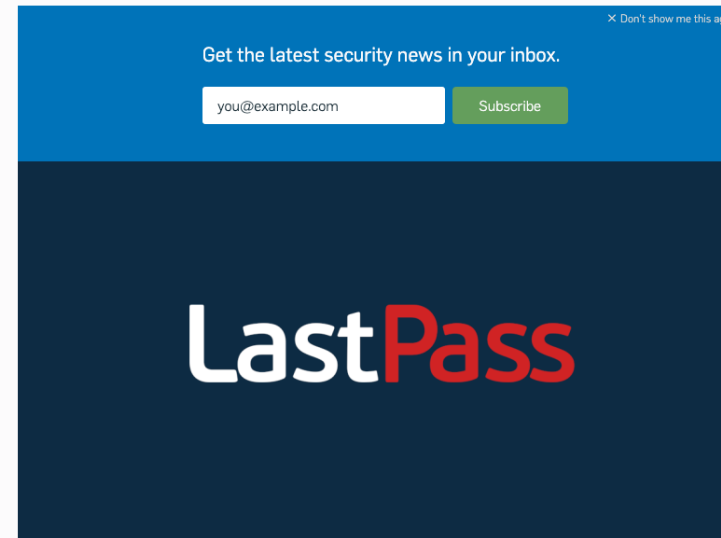
By Jill McKeon



August 29, 2022 - LastPass, a password manager with 25 million users around the world, disclosed a data breach to its customers. After detecting unusual activity in early August, LastPass discovered that an unauthorized party had stolen portions of source code and some proprietary technical information.

LastPass source code breach – incident response report released

19 SEP 2022 0
Data loss



Previous: S3 Ep100.5: Uber breach – an expert speaks [...]

by Paul Ducklin



If the big story of this month looks set to be [Uber's data breach](#), where a hacker was allegedly able to roam widely through the ride-sharing company's network...

“Our investigation determined that the threat actor gained access to the Development environment using a developer’s compromised endpoint.

While the method used for the initial endpoint compromise is inconclusive, the threat actor utilized their persistent access to impersonate the developer once the developer had successfully authenticated using multi-factor authentication.”

Anyone is using LastPass? 1password?
Why or Why not?
How is it related to SSD?

DeepSeek iOS Mobile App

NowSecure has conducted a comprehensive security and privacy assessment of the DeepSeek iOS mobile app, uncovering multiple critical vulnerabilities that put individuals, enterprises, and government agencies at risk. These findings highlight the immediate need for organizations to prohibit the app's use to safeguard sensitive data and mitigate potential cyber risks.

Unencrypted Data Transmission: The app transmits sensitive data over the internet without encryption, making it vulnerable to interception and manipulation.

Weak & Hardcoded Encryption Keys: Uses outdated Triple DES encryption, reuses initialization vectors, and hardcodes encryption keys, violating best security practices.

Insecure Data Storage: Username, password, and encryption keys are stored insecurely, increasing the risk of credential theft.

Extensive Data Collection & Fingerprinting: The app collects user and device data, which can be used for tracking and de-anonymization.

Data Sent to China & Governed by PRC Laws: User data is transmitted to servers controlled by ByteDance, raising concerns over government access and compliance risks.

NowSecure Uncovers Multiple Security and Privacy Flaws in DeepSeek iOS Mobile App

Posted by [Andrew Hoog](#) ✓



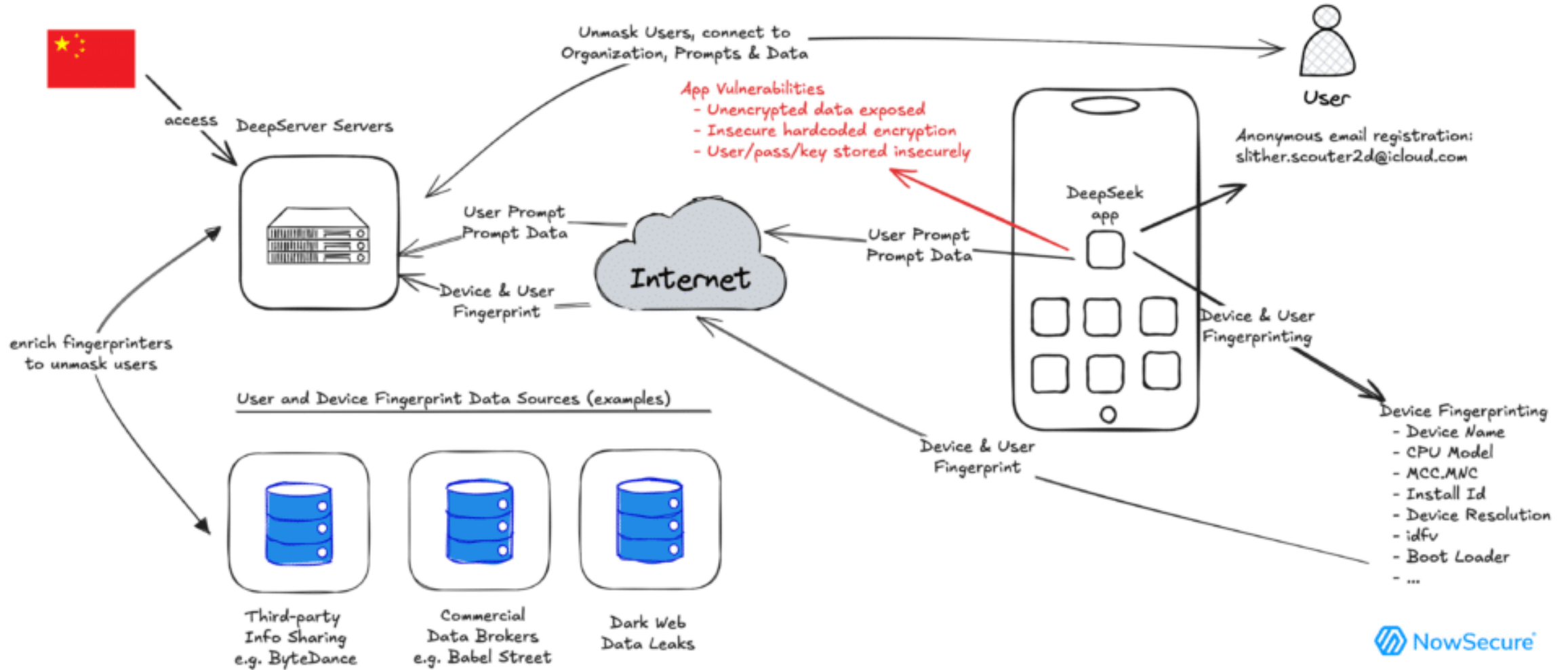
FEBRUARY 06, 2025 | [Mobile Security Solutions](#), [Research & Threat Intel](#)

SHARE    

A NowSecure [mobile application security and privacy assessment](#) has uncovered multiple [security and privacy issues](#) in the DeepSeek iOS mobile app that lead us to urge enterprises to prohibit/forbid its usage in their organizations.

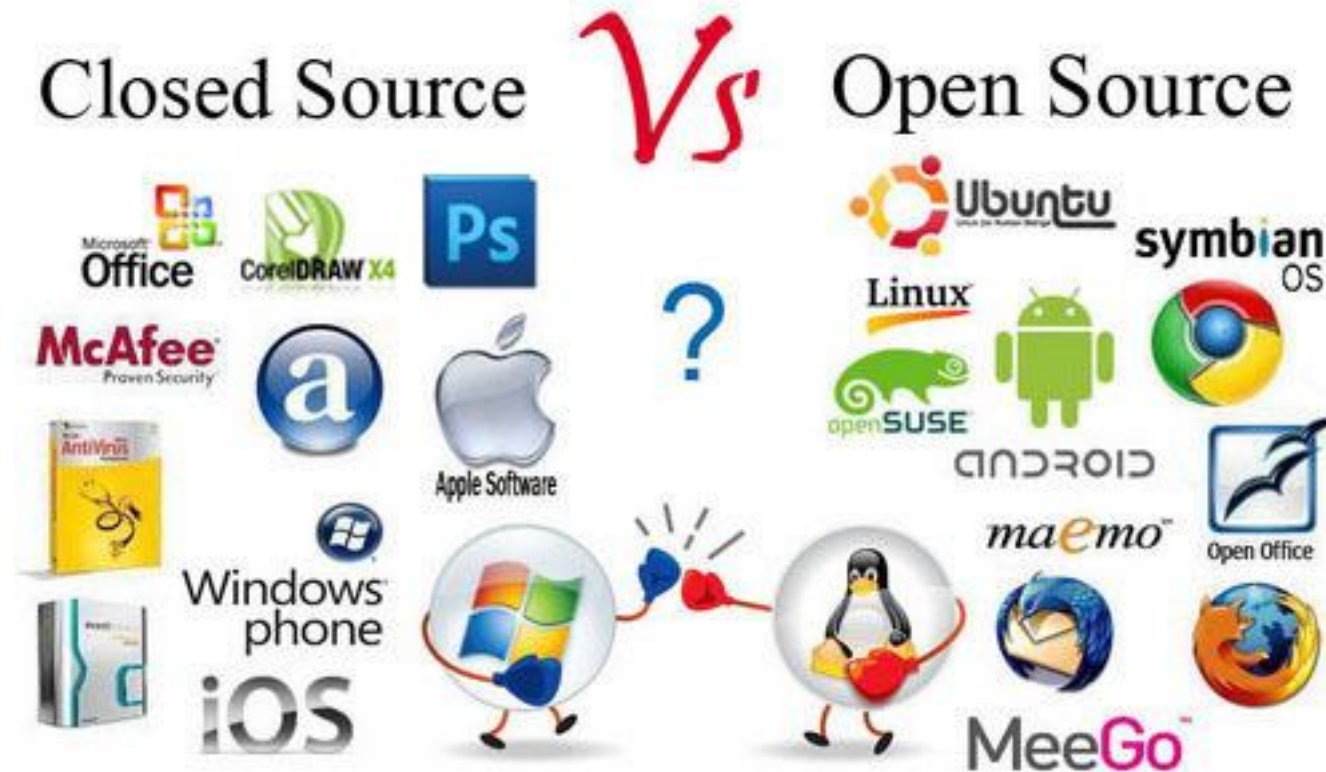
As the [top iOS app](#) since [Jan 25, 2025](#), the DeepSeek iOS app has already been downloaded and used on millions of devices belonging to individuals enterprise and government employees, prompting [swift bans](#) from countries, state and federal governments and the U.S. military to protect their organizations and safeguard national security.

DeepSeek iOS Mobile App



Open-source vs Closed-source

- Which one you prefer?
- Which one is more “secure”?



Agenda

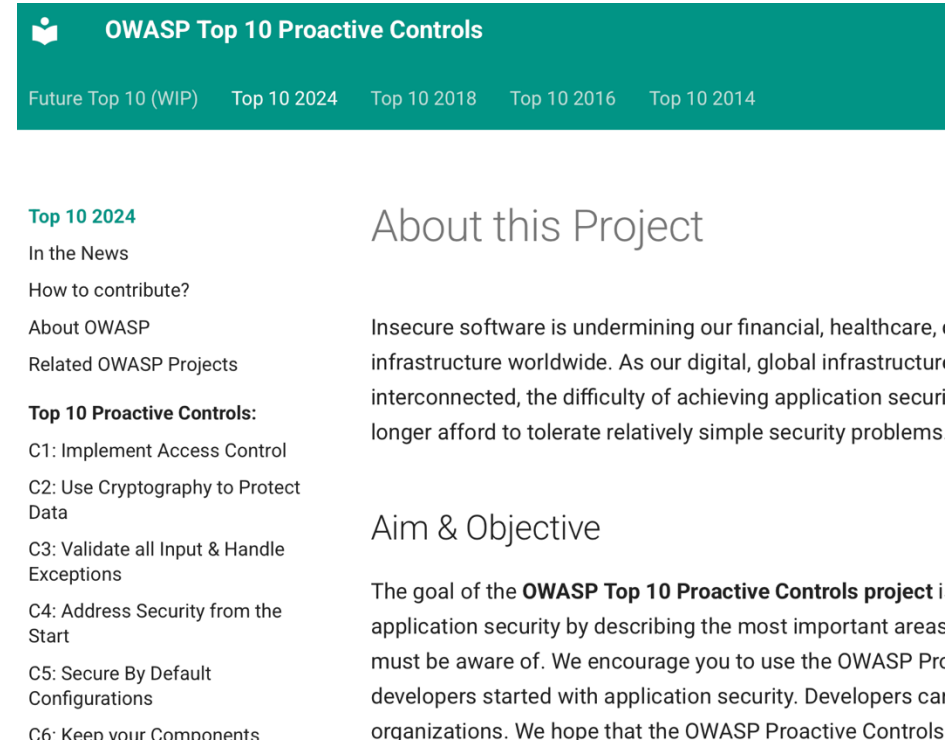
OWASP

- Top 10 Proactive Controls
 - <https://owasp.org/www-project-proactive-controls/>
- Cheat Sheet Series
 - <https://cheatsheetseries.owasp.org/>
- Security Knowledge Framework
 - <https://demo.securityknowledgeframework.org/>
- Secure Coding Practices Quick Reference Guide
 - https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/migrated_content

And more ...

OWASP Top 10 Proactive Controls

- Describes the most important security controls that **should be adopted** in every secure software development project
- The OWASP Top 10 Proactive Controls 2024 is similar to the OWASP Top 10 but is focused on defensive techniques and controls as opposed to risks.
 - C1: Implement Access Control
 - **C2: Use Cryptography to Protect Data**
 - **C3: Validate all Input & Handle Exceptions**
 - C4: Address Security from the Start
 - C5: Secure By Default Configurations
 - **C6: Keep your Components Secure**
 - C7: Secure Digital Identities
 - C8: Leverage Browser Security Features
 - **C9: Implement Security Logging and Monitoring**
 - C10: Stop Server Side Request Forgery



The screenshot shows the OWASP Top 10 Proactive Controls website. The header is teal with the title "OWASP Top 10 Proactive Controls" and a navigation bar with links: "Future Top 10 (WIP)", "Top 10 2024", "Top 10 2018", "Top 10 2016", and "Top 10 2014". The main content area is divided into two columns. The left column, titled "Top 10 2024", lists links for "In the News", "How to contribute?", "About OWASP", "Related OWASP Projects", and a list of the first six controls (C1 to C6). The right column, titled "About this Project", contains a paragraph about insecure software and a section titled "Aim & Objective" which states the goal of the project is to describe the most important areas of application security.

OWASP Top 10 Proactive Controls

Future Top 10 (WIP) Top 10 2024 Top 10 2018 Top 10 2016 Top 10 2014

Top 10 2024

In the News

How to contribute?

About OWASP

Related OWASP Projects

Top 10 Proactive Controls:

C1: Implement Access Control

C2: Use Cryptography to Protect Data

C3: Validate all Input & Handle Exceptions

C4: Address Security from the Start

C5: Secure By Default Configurations

C6: Keep your Components

About this Project

Insecure software is undermining our financial, healthcare, and infrastructure worldwide. As our digital, global infrastructure becomes more interconnected, the difficulty of achieving application security will no longer afford to tolerate relatively simple security problems.

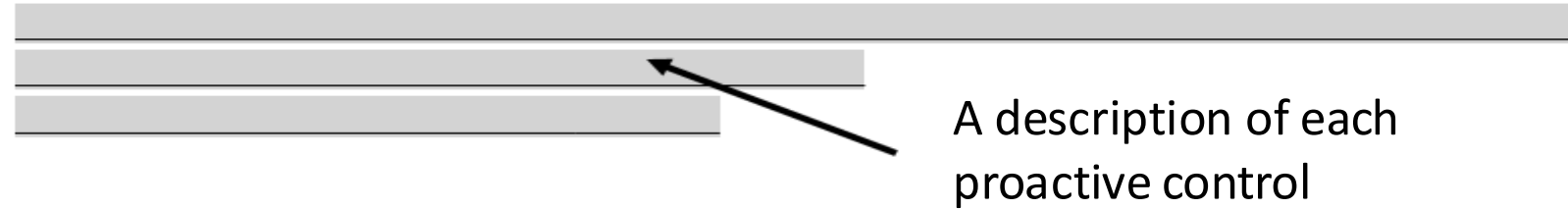
Aim & Objective

The goal of the **OWASP Top 10 Proactive Controls project** is to describe the most important areas of application security by describing the most important areas of application security that developers must be aware of. We encourage you to use the OWASP Proactive Controls if your organization started with application security. Developers can use the OWASP Proactive Controls to help their organizations. We hope that the OWASP Proactive Controls

OWASP Top 10 Proactive Controls

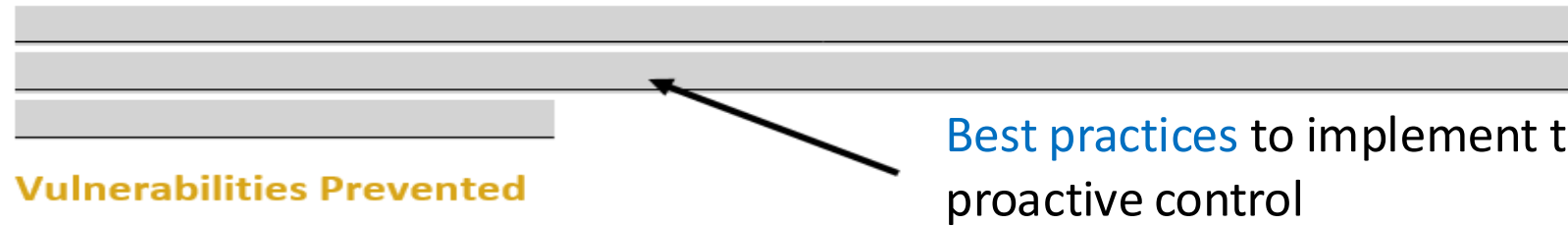
- OWASP Top 10 Proactive Controls serves as the **starting point** to building **secure software** with recommended **implementation best practices** and **tools**.

Description



A description of each proactive control

Implementation



Best practices to implement the proactive control

Vulnerabilities Prevented

-
-

Vulnerabilities or risks addressed by the proactive control

References

-
-

References for further study

Tools

-
-

Tools to introduce/integrate into your secure software development

C2: Use Cryptography to Protect Data

Description: Sensitive data such as passwords, credit card numbers, health records, personal information and business secrets require extra protection, particularly if that data falls under privacy laws (EU's General Data Protection Regulation **GDPR**), financial data protection rules such as PCI Data Security Standard (**PCI DSS**) or other regulations.

Implementation Best Practices:

- **Never transmit plain-text data.** The technical capability exists to easily encrypt all data that is sent between any point A and B. Embrace the use of cryptography to protect all data at rest and in transit.
- **Do not create your own cryptographic protocols.** The creation of a cryptographic protocol is a tricky proposition. When **NIST created AES, they had an open competition where the best cryptographers worldwide submitted proposals** and then looked for flaws in the other proposals. Instead of using your developer cycles to create a new crypto protocol, use an existing, battle-tested standard. Focus your innovation on making your feature or product better.
- **Do not implement cryptographic routines.** Use an existing library that implements cryptographic routines.

C2: Use Cryptography to Protect Data

Vulnerabilities Prevented:

- https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
- <https://mas.owasp.org/MASVS/controls/MASVS-CRYPTO-1/>

Tools:

- <https://github.com/nabla-c0d3/sslyze>
- <https://testssl.sh/>
- SSLyze - SSL configuration scanning library and CLI tool
- SSLabs - Free service for scanning and checking TLS/SSL configuration
- OWASP O-Saft TLS Tool - TLS connection testing tool
- GitRob - Command line tool to find sensitive information in publicly available files on GitHub

C3: Validate all Input & Handle Exception

Description:

Input validation is a programming technique that ensures only properly formatted data may enter a software system component. When the injection attack targets a client (for example JavaScript based attacks), web servers can perform quoting/encoding on the attacker-provided data before forwarding it to the client.

Injection attacks commonly occur if an application confuses data input as executable commands and are often possible where input validation is forgotten or implemented wrong. For example, imagine that a web application accepts an email address as input from a user. The email address would be the expected “data”. Attackers now search for ways to confuse applications to execute this (supposed) data as commands. Different injection attacks target different areas:

- When an attacker tricks an application into **interpreting user input (data) as SQL commands (or parts thereof)**, there is a SQL injection attack. The injected command executes within the database server.
- **Remote command injection (RCE)** happens if an application confuses user data with commands that execute on the web application server/host. Server-Side Template Injections are another example of injections executed within the application server.
- When **JavaScript-Injections** happen, a web application has accepted user data but is coerced to execute that data as code. Injected JavaScript code is typically executed within another user’s web browser, thus not directly attacking the web server but other users.

C3: Validate all Input & Handle Exception

Implementation:

- Allowlisting vs Denylisting
- Client-side and Server-side Validation
- Regular Expressions

References

- OWASP Cheat Sheet: Input Validation
- OWASP Testing Guide: Testing for Input Validation
- OWASP Cheat Sheet Series: DOM based XSS Prevention
- Injection Prevention Cheat Sheet
- Hardening with CSP: CSP with Google

Tools

- OWASP Java HTML Sanitizer Project
- Java JSR-303/JSR-349 Bean Validation
- Java Hibernate Validator Apache Commons Validator PHP's filter functions
- Sqlmap.py
- ZAP-based scans
- CSP Evaluator

C6: Keep your Components Secure

Description:

It is a common practice in software development to leverage libraries and frameworks. Secure libraries and software frameworks with embedded security help software developers prevent security-related design and implementation flaws.

A developer writing an application from scratch **might not have sufficient knowledge, time, or budget to properly implement or maintain security features**. Leveraging security frameworks (both open source and vendor) help accomplish security goals more efficiently and accurately.

When possible, the emphasis should be on using the existing secure features of frameworks rather than importing yet another third party libraries, which **requires regular updates and maintenance**. It is preferable to have developers take advantage of what they're already using instead of forcing yet another library on them.

When incorporating third party libraries or frameworks into your software, it is important to consider the following two categories of best practices:

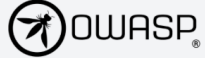
1. **Identify trusted libraries and frameworks to bring into your software.**
2. **Monitor and update packages to ensure that your software is not vulnerable to the possible security vulnerabilities introduced by the third party components.**

C6: Keep your Components Secure

Tools:

- **OWASP Dependency-Check** - to identify project dependencies and check for publicly disclosed vulnerabilities
- OWASP Dependency-Track - periodically monitor SBOM files for new vulnerabilities
- RetireJS scanner for JavaScript libraries
- Renovate for automated dependencies updates
- Harbor : an open source registry that secures artifacts with policies and role-based access control

OWASP Dependency Check

PROJECTS CHAPTERS EVENTS ABOUT

[Store](#)

[Donate](#)

[Join](#)

OWASP Dependency-Check

Dependency-Check is a Software Composition Analysis (SCA) tool that attempts to detect publicly disclosed vulnerabilities contained within a project's dependencies. It does this by determining if there is a Common Platform Enumeration (CPE) identifier for a given dependency. If found, it will generate a report linking to the associated CVE entries.

Introduction

The OWASP Top 10 2013 contains a new entry: A9-Using Components with Known Vulnerabilities. Dependency Check can currently be used to scan applications (and their dependent libraries) to identify any known vulnerable components.

The problem with using known vulnerable components was described very well in a paper by Jeff Williams and Arshan Dabirsiaghi titled, "[Unfortunate Reality of Insecure Libraries](#)". The gist of the paper is that we as a development community include third party libraries in our applications that contain well known published vulnerabilities (such as those at the [National Vulnerability Database](#)).

Dependency-check has a command line interface, a Maven plugin, an Ant task, and a Jenkins plugin. The core engine contains a series of analyzers that inspect the project dependencies, collect pieces of information about the dependencies (referred to as evidence within the tool). The evidence is then used to identify the [Common Platform Enumeration \(CPE\)](#) for the given dependency. If a CPE is identified, a listing of associated [Common Vulnerability and Exposure \(CVE\)](#) entries are listed in a report. Other 3rd party services and data sources such as the NPM Audit API, the OSS Index, RetireJS, and Bundler Audit are utilized for specific technologies.

Dependency-check automatically updates itself using the [NVD Data Feeds](#) hosted by NIST. "IMPORTANT NOTE:" The initial download of the data may take ten minutes or more. If you run the tool at least once every seven days, only a small JSON file needs to be downloaded to keep the local copy of the data current.

[Watch](#) 163

[Star](#) 4,396

The OWASP® Foundation works to improve the security of software through its community-led open source software projects, hundreds of chapters worldwide, tens of thousands of members, and by hosting local and global conferences.

Project Classification



[Builders](#) [Defenders](#)

[OWASP TOOL PROJECT](#)

Downloads

C9: Implement Security Logging and Monitoring

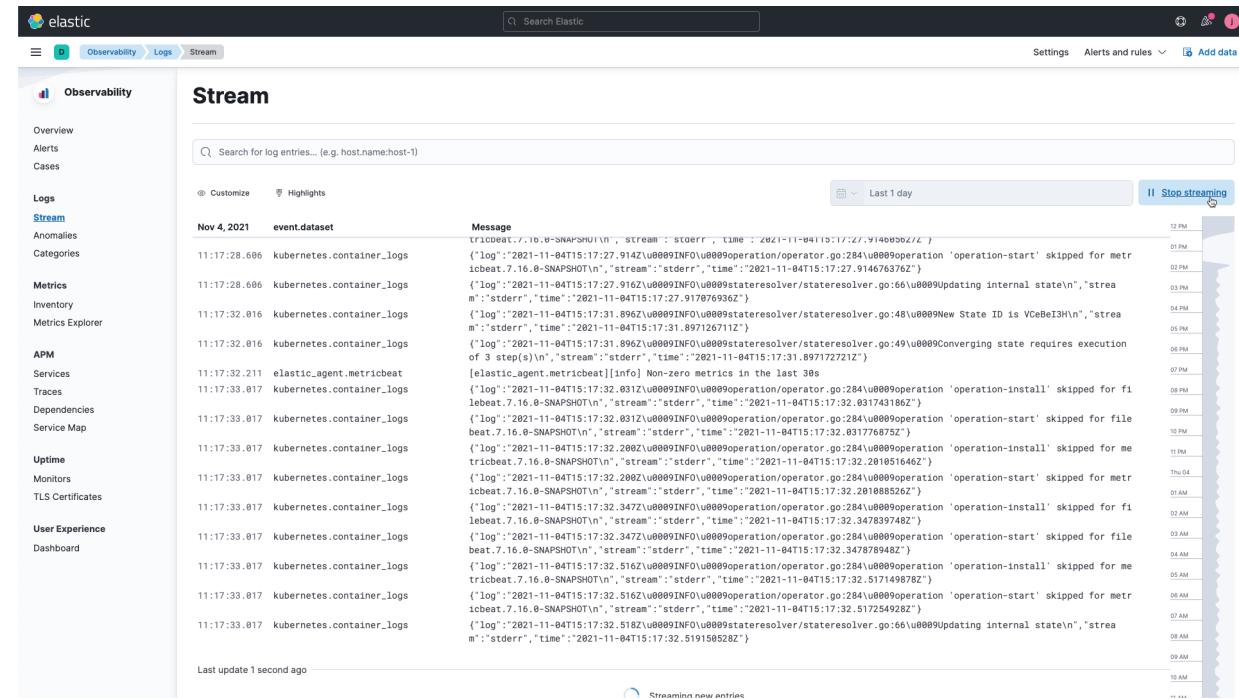
Logging: Debugging and diagnostic purposes.

Security logging: To log security information during the runtime operation of an application.

Monitoring is the live review of application and security logs using various forms of automation. The same tools and patterns can be used for operations, debugging and security purposes.

Security logging can be used for:

- Feeding intrusion detection systems
- Forensic analysis and investigations
- Satisfying regulatory compliance requirements



Secure Logging design and implementation

- **Encode** and **validate** any dangerous characters before logging to prevent log injection attacks.
- **Do not log sensitive information**. For example, do not log password, session ID, credit cards, or social security numbers.
- Protect log **integrity**. An attacker may attempt to tamper with the logs. Therefore, the permission of log files and log changes audit should be considered.
- Forward logs from distributed systems to a **central, secure logging service**. This will ensure log data cannot be lost if one node is compromised. This also allows for centralized monitoring.
- PDPA!

Secure Logging Library

- Log4J?
- winston
- Pino

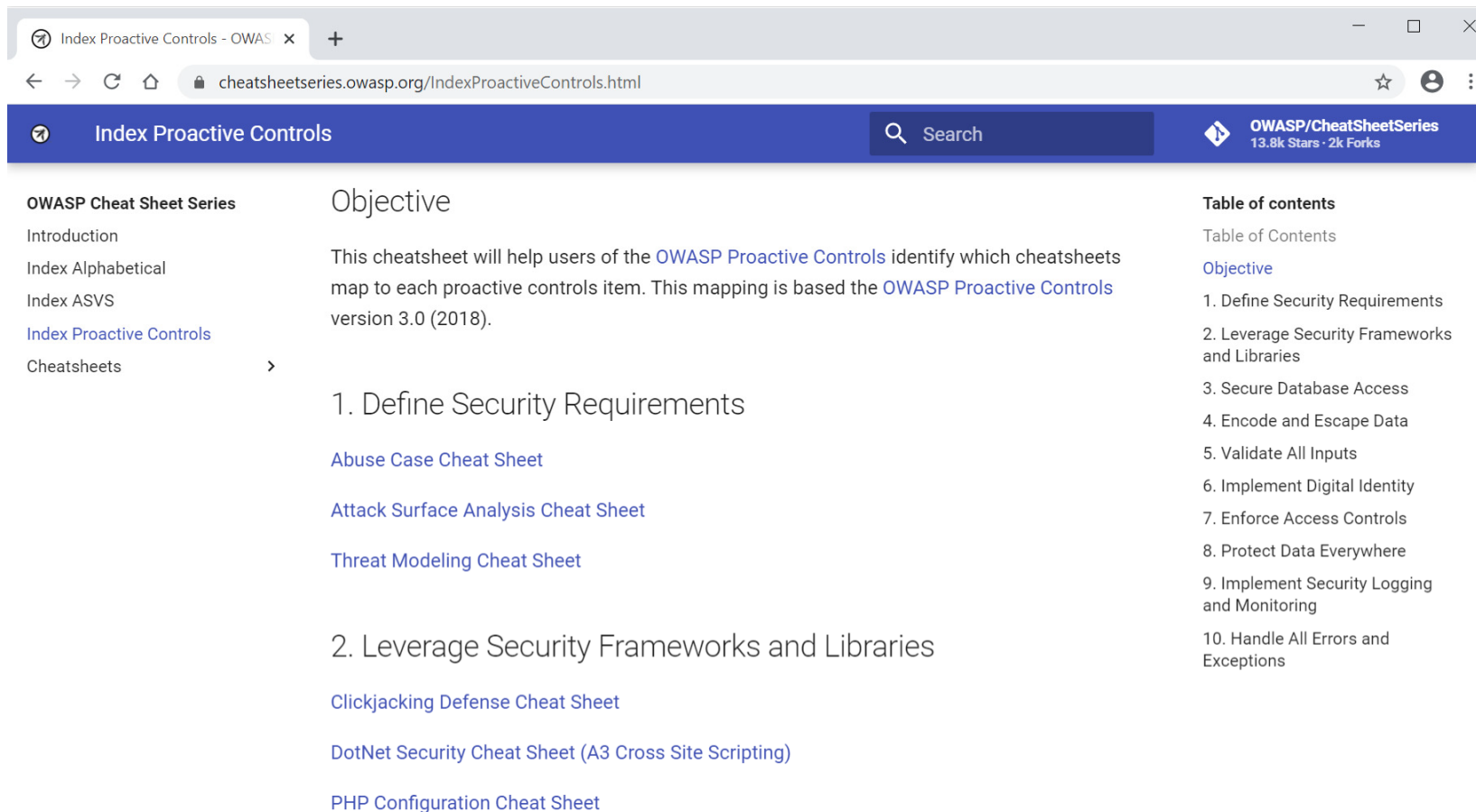
And more...



OWASP Cheat sheet series

OWASP Cheat Sheet Series

- It was created by various application security professionals to provide **security guidance** on specific security topics.



The screenshot shows a web browser window with the URL `cheatsheetseries.owasp.org/IndexProactiveControls.html`. The page title is "Index Proactive Controls". The left sidebar contains a navigation menu with the following items: "OWASP Cheat Sheet Series", "Introduction", "Index Alphabetical", "Index ASVS", "Index Proactive Controls" (highlighted), and "Cheatsheets". The main content area is titled "Objective" and contains the text: "This cheatsheet will help users of the [OWASP Proactive Controls](#) identify which cheatsheets map to each proactive controls item. This mapping is based the [OWASP Proactive Controls](#) version 3.0 (2018).". Below this, there are three sections: "1. Define Security Requirements" with links to "Abuse Case Cheat Sheet", "Attack Surface Analysis Cheat Sheet", and "Threat Modeling Cheat Sheet"; "2. Leverage Security Frameworks and Libraries" with links to "Clickjacking Defense Cheat Sheet", "DotNet Security Cheat Sheet (A3 Cross Site Scripting)", and "PHP Configuration Cheat Sheet"; and "3. Secure Database Access" (partially visible). The right sidebar contains a "Table of contents" with links to "Table of Contents", "Objective", and a list of 10 numbered items: "1. Define Security Requirements", "2. Leverage Security Frameworks and Libraries", "3. Secure Database Access", "4. Encode and Escape Data", "5. Validate All Inputs", "6. Implement Digital Identity", "7. Enforce Access Controls", "8. Protect Data Everywhere", "9. Implement Security Logging and Monitoring", and "10. Handle All Errors and Exceptions".

OWASP Cheat Sheet Series

- In total there are 78 cheat sheets (Keep increasing)
- For this lecture, we will focus on **Input Validation** Cheat Sheet and **SQL Injection** Prevention Cheat Sheet

Index Alphabetical



78 cheat sheets available.

Icons beside the cheat sheet name indicate in which language(s) code snippet(s) are provided.

A B C D E F G H I J K L M N O P Q R S T U V W X

A

[AJAX Security Cheat Sheet.](#) (JSON)

[Access Control Cheat Sheet.](#)

[Authorization Testing Automation Cheat Sheet.](#)



[Attack Surface Analysis Cheat Sheet.](#)

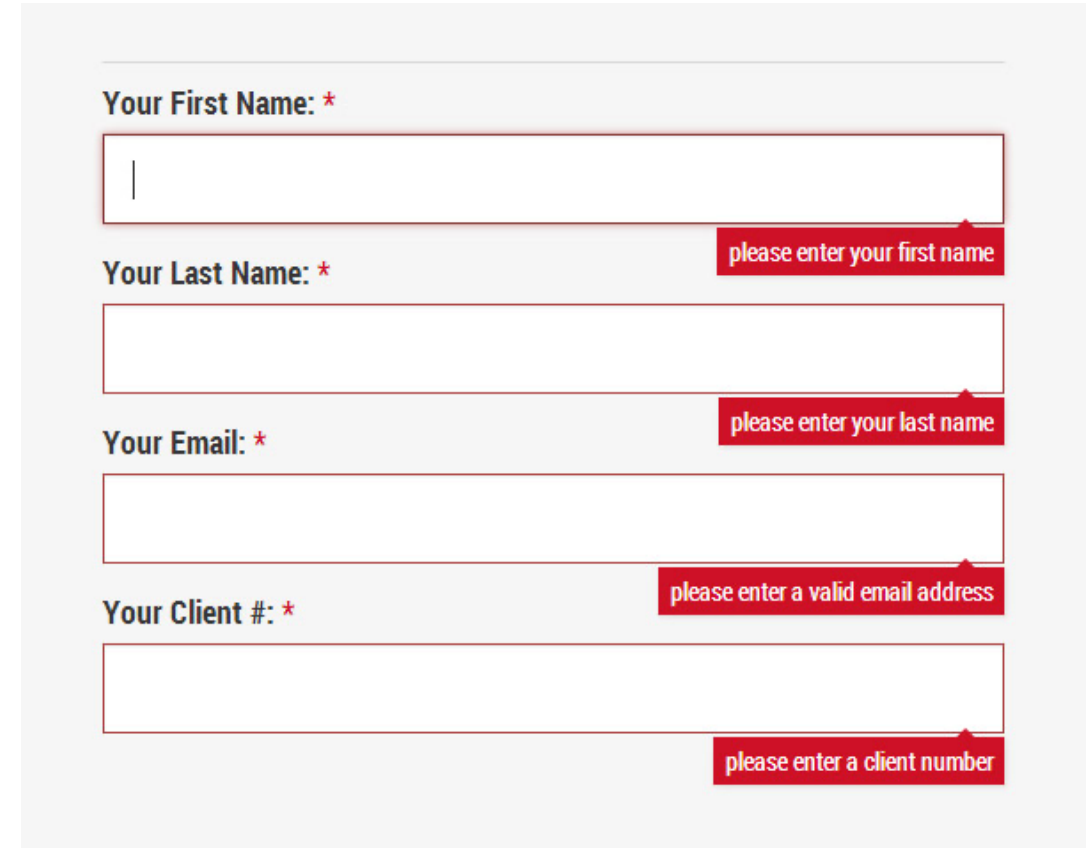
[Authorization Cheat Sheet.](#)

[Authentication Cheat Sheet.](#)

[Abuse Case Cheat Sheet.](#)

Input Validation Cheat Sheet

- Input validation is performed to ensure only properly formed data is entering the workflow in an information system, **preventing malformed data** from persisting in the database and triggering malfunction of various downstream components.
- It should happen **as early as possible** in the data flow, preferably as soon as the data is received from the external party.
- It should **not** be used as the primary method of preventing **XSS, SQL Injection and other attacks**.



Your First Name: *

please enter your first name

Your Last Name: *

please enter your last name

Your Email: *

please enter a valid email address

Your Client #: *

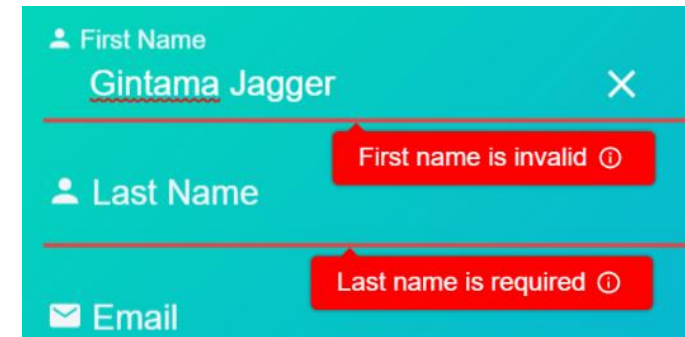
please enter a client number

Implementing input validation

- Data type validators available natively in web application frameworks (such as **Django Validators**, **Apache Commons Validators** etc).
- Validation against **JSON Schema** and **XML Schema (XSD)** for input in these formats.
- **Type conversion** (e.g. `Integer.parseInt()` in Java, `int()` in Python) with strict exception handling
- Minimum and maximum value **range check** for numerical parameters and dates, minimum and maximum length check for strings.
- Array of allowed values for small sets of string parameters (e.g. days of week).
- **Regular expressions** for any other structured data covering the whole input string (`^...$`) and not using "any character" wildcard (such as `.` or `\S`)

Allow list vs block list

- It is a common mistake to use block list validation in order to try to detect possibly dangerous characters and patterns like the apostrophe ' character, the string 1=1, or the <script> tag, but this is a massively flawed approach as it is trivial for an attacker to bypass such filters.
- Plus, such filters frequently prevent authorized input, like O'Brian, where the ' character is fully legitimate.
- Allow list validation is appropriate for all input fields provided by the user. Allow list validation involves defining exactly what IS authorized, and by definition, everything else is not authorized.
- If it's well structured data, like dates, social security numbers, zip codes, email addresses, etc. then the developer should be able to define a very strong validation pattern, usually based on regular expressions, for validating such input.
- If the input field comes from a fixed set of options, like a drop-down list or radio buttons, then the input needs to match exactly one of the values offered to the user in the first place.



A screenshot of a registration form with a teal background. It contains three input fields: 'First Name' with the value 'Gintama Jagger', 'Last Name', and 'Email'. A red error message 'First name is invalid ⓘ' is shown next to the First Name field. Another red error message 'Last name is required ⓘ' is shown next to the Last Name field. A close button 'X' is visible in the top right corner of the form area.

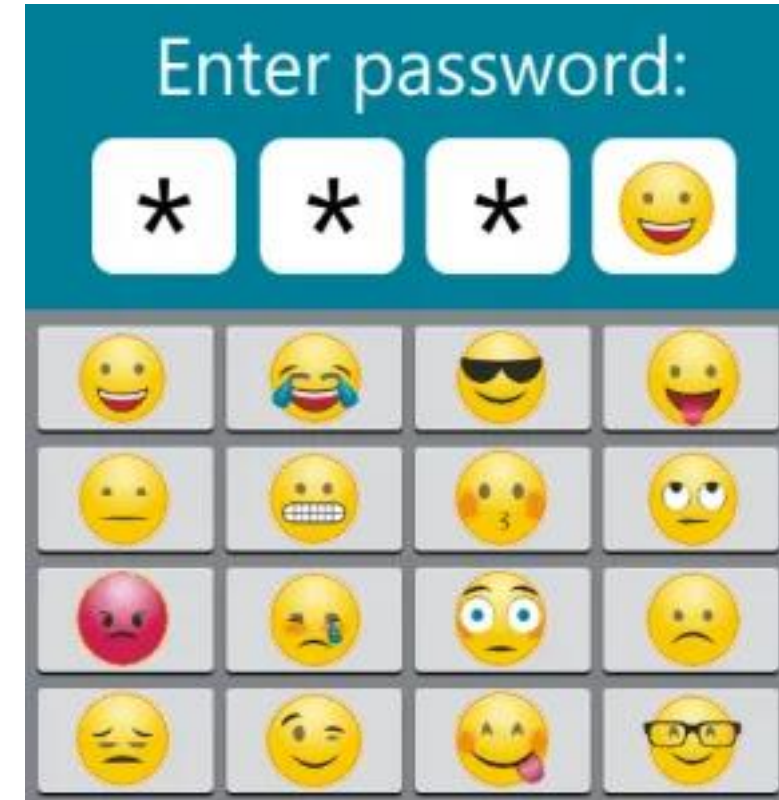
Validating free-form Unicode text

Free-form text, especially with Unicode characters, is perceived as difficult to validate due to a relatively large space of characters that need to be allowed.

It's also free-form text input that highlights the importance of proper context-aware output encoding and quite clearly demonstrates that input validation is **not** the primary safeguards against Cross-Site Scripting.

The primary means of input validation for free-form text input should be:

- **Normalization:** Ensure canonical encoding is used across all the text and no invalid characters are present.
- **Character category allow-listing:** Unicode allows listing categories such as "decimal digits" or "letters" which not only covers the Latin alphabet but also various other scripts used globally (e.g. Arabic, Cyrillic, CJK ideographs etc).
- **Individual character allow-listing:** If you allow letters and ideographs in names and also want to allow apostrophe ' for Irish names, but don't want to allow the whole punctuation category.



Your application support emoji?

Regular expressions

- Developing regular expressions can be complicated, and is well beyond the scope of this cheat sheet.
- There are lots of resources on the internet about how to write regular expressions, including the cheatsheet and the [OWASP Validation Regex Repository](#).
- When designing regular expression, be aware of [RegEx Denial of Service \(ReDoS\) attacks](#). These attacks cause a program using a poorly designed Regular Expression to operate very slowly and utilize CPU resources for a very long time.

In summary, input validation should:

- Be applied to all input data, at minimum.
- Define the allowed set of characters to be accepted.
- Define a minimum and maximum length for the data (e.g. {1,25}).

Allow List Regular Expression Examples

Validating a U.S. Zip Code (5 digits plus optional -4)

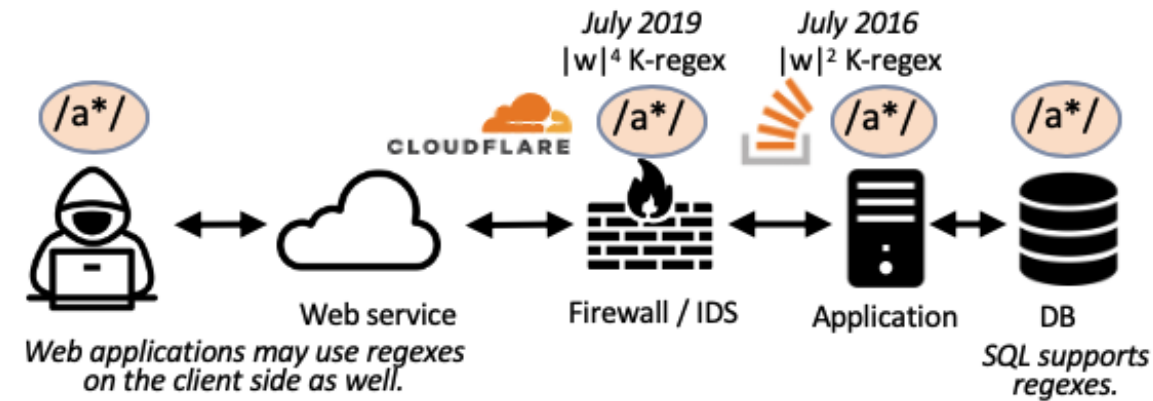
- `^\d{5}(-\d{4})?$`

Validating U.S. State Selection From a Drop-Down Menu

- `^(AA|AE|AP|AL|AK|AS|AZ|AR|CA|CO|CT|DE|DC|FM|FL|GA|GU|HI|ID|IL|IN|IA|KS|KY|LA|ME|MH|MD|MA|MI|MN|MS|MO|MT|NE|NV|NH|NJ|NM|NY|NC|ND|MP|OH|OK|OR|PW|PA|PR|RI|SC|SD|TN|TX|UT|VT|VI|VA|WA|WV|WI|WY)$`

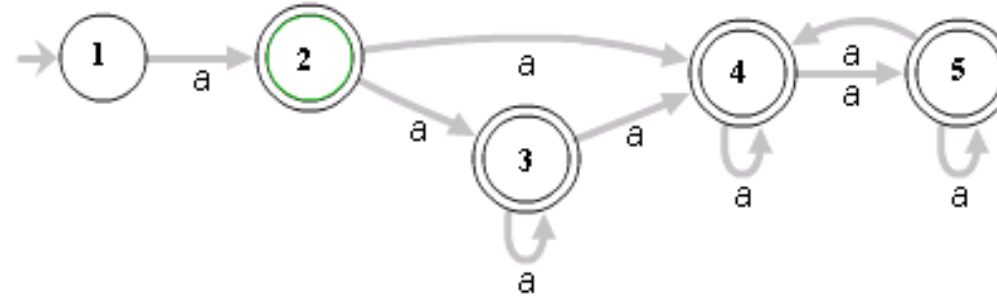
Regular expression Denial of Service - ReDoS

- The Regular expression Denial of Service (ReDoS) is a Denial of Service attack, that exploits the fact that most Regular Expression implementations may reach extreme situations that cause them to work very slowly (exponentially related to input size).
- An attacker can then cause a program using a Regular Expression (Regex) to enter these extreme situations and then hang for a very long time.



Regular expression Denial of Service - ReDoS

For example, the Regex pattern or quantifier $\text{^(a+)+\$}$ is represented by the following non-deterministic finite automata (NFA):



- For the input **aaaaX** there are **16** possible paths in the above graph. But for **aaaaaaaaaaaaaaaaaaaaX** there are **65536** possible paths, and the number is double for each additional **a**. This is an extreme case where the naïve algorithm is problematic, because it must pass on many paths to find a **non-matching input**.
- The root-cause of the above example is in a Regex engine feature called **backtracking**. Simply, if the input (token) fails to match, the engine goes back to previous positions where it could take a different path. The engine tries this many times until it explores all possible paths. In the above example, this feature create a long running loop because there were many paths to explore due to inefficient Regex pattern.

Evil Regex

A Regex pattern is called **Evil Regex** if it can get stuck on crafted input.

Evil Regex contains:

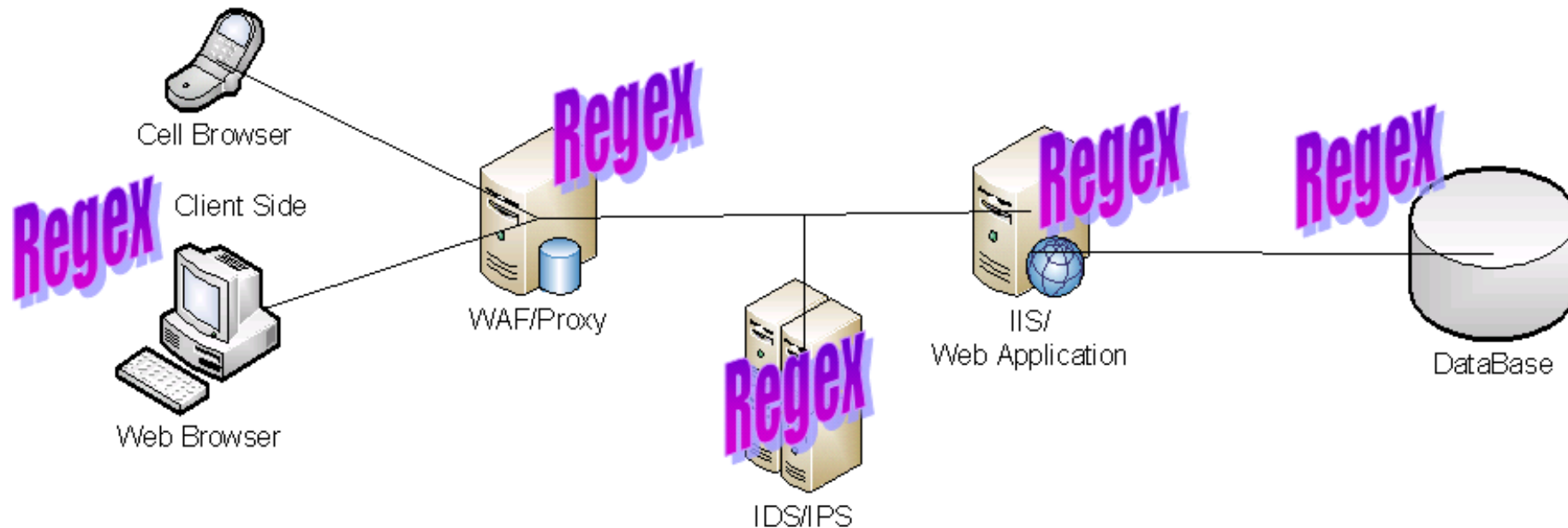
- Grouping with repetition
- Inside the repeated group:
 - Repetition
 - Alternation with overlapping

Examples of Evil Regex:

- `(a+)+`
- `([a-zA-Z]+)*`
- `(a|aa)+`
- `(a|a?)+`
- `(.*a){x}` for $x \gg 10$
- All the above are susceptible to the input **aaaaaaaaaaaaaaaaaaaaaaaaaaaaa!** (The minimum input length might change slightly, when using faster or slower machines).

Evil Regex attack

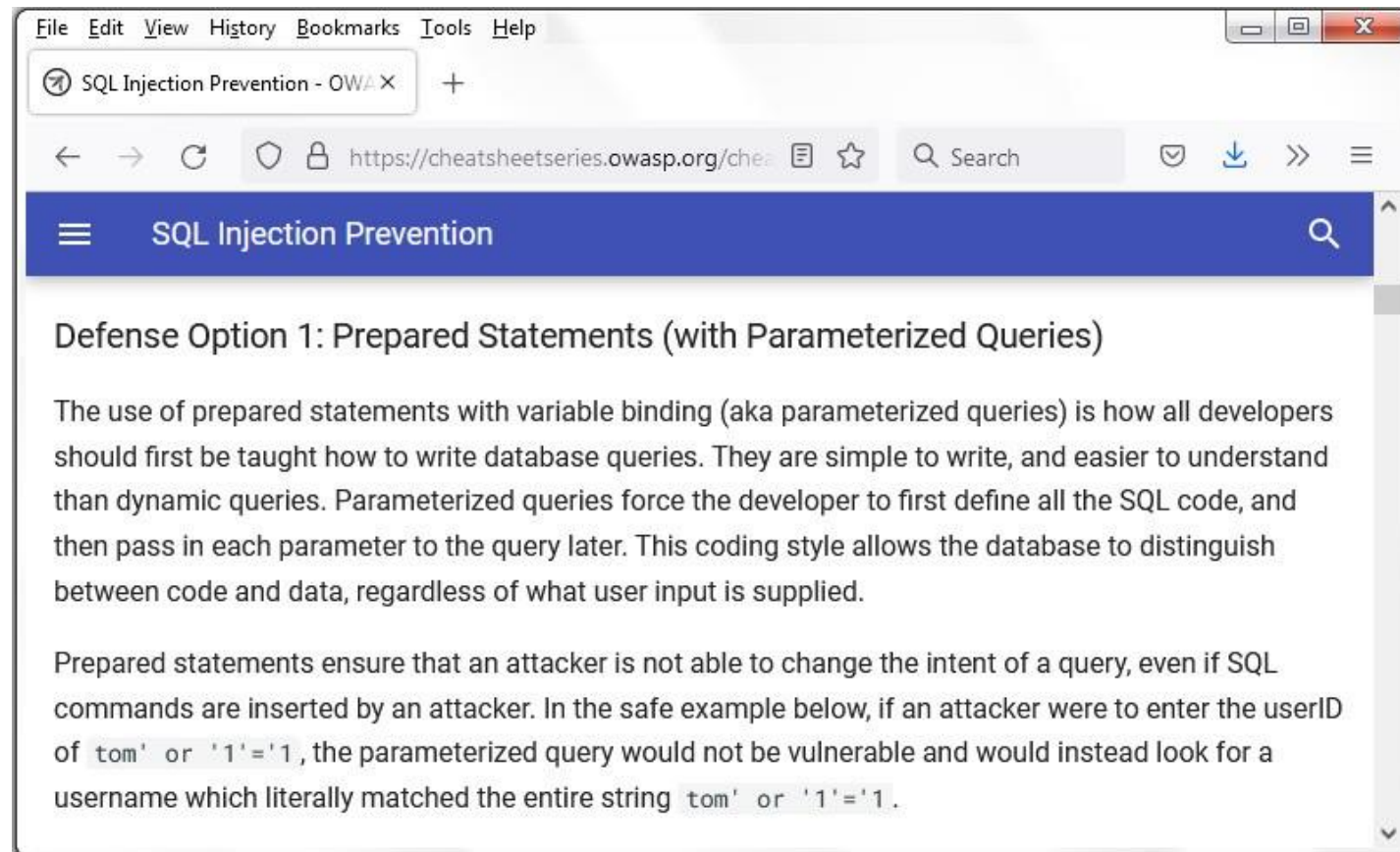
- The attacker might use the above knowledge to look for applications that use Regular Expressions, containing an **Evil Regex**, and send a well-crafted input, that will hang the system.
- Alternatively, if a Regex itself is affected by a user input, the attacker can inject an **Evil Regex**, and make the system vulnerable.
- An attacker can hang a WEB-browser (on a computer or potentially also on a mobile device), hang a Web Application Firewall (WAF), attack a database, and even stack a vulnerable WEB server.



SQL Injection Prevention Cheat Sheet

- Defenses to prevent **SQL injection** in **OWASP Cheat Sheet Series**:

<https://cheatsheetseries.owasp.org/cheatsheets/SQL Injection Prevention Cheat Sheet.html>



SQL Injection Prevention Cheat Sheet

Primary Defenses:

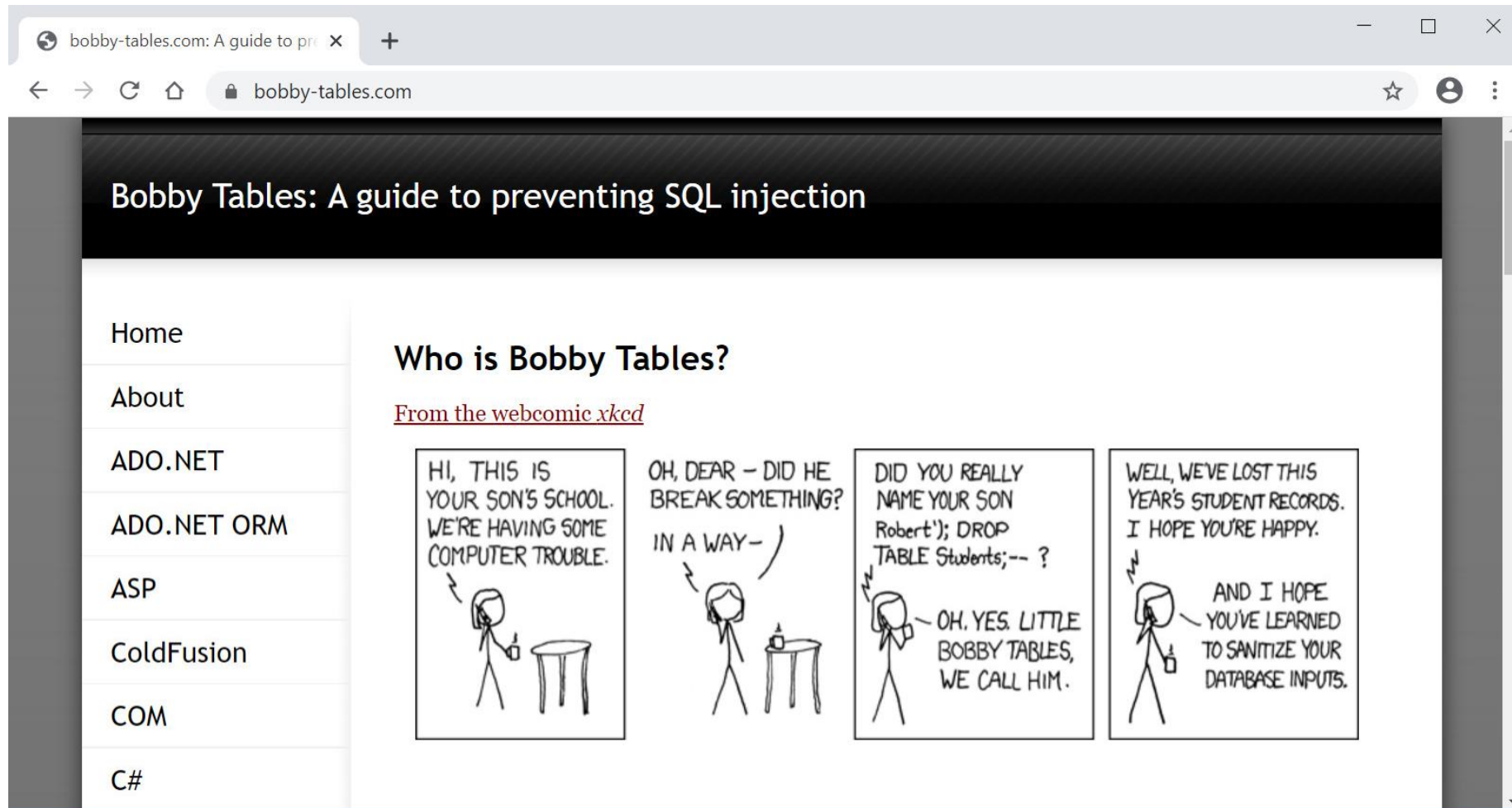
- Option 1: Use of **Prepared Statements** (with **Parameterized Queries**)
- Option 2: Use of Properly Constructed **Stored Procedures**
- Option 3: Allow-list Input Validation
- Option 4: Escaping All User Supplied Input

Additional Defenses:

- Also: Enforcing Least Privilege
- Also: Performing Allow-list Input Validation as a Secondary Defense

Bobby Tables

- **Bobby Tables** is a guide to preventing **SQL injection**.



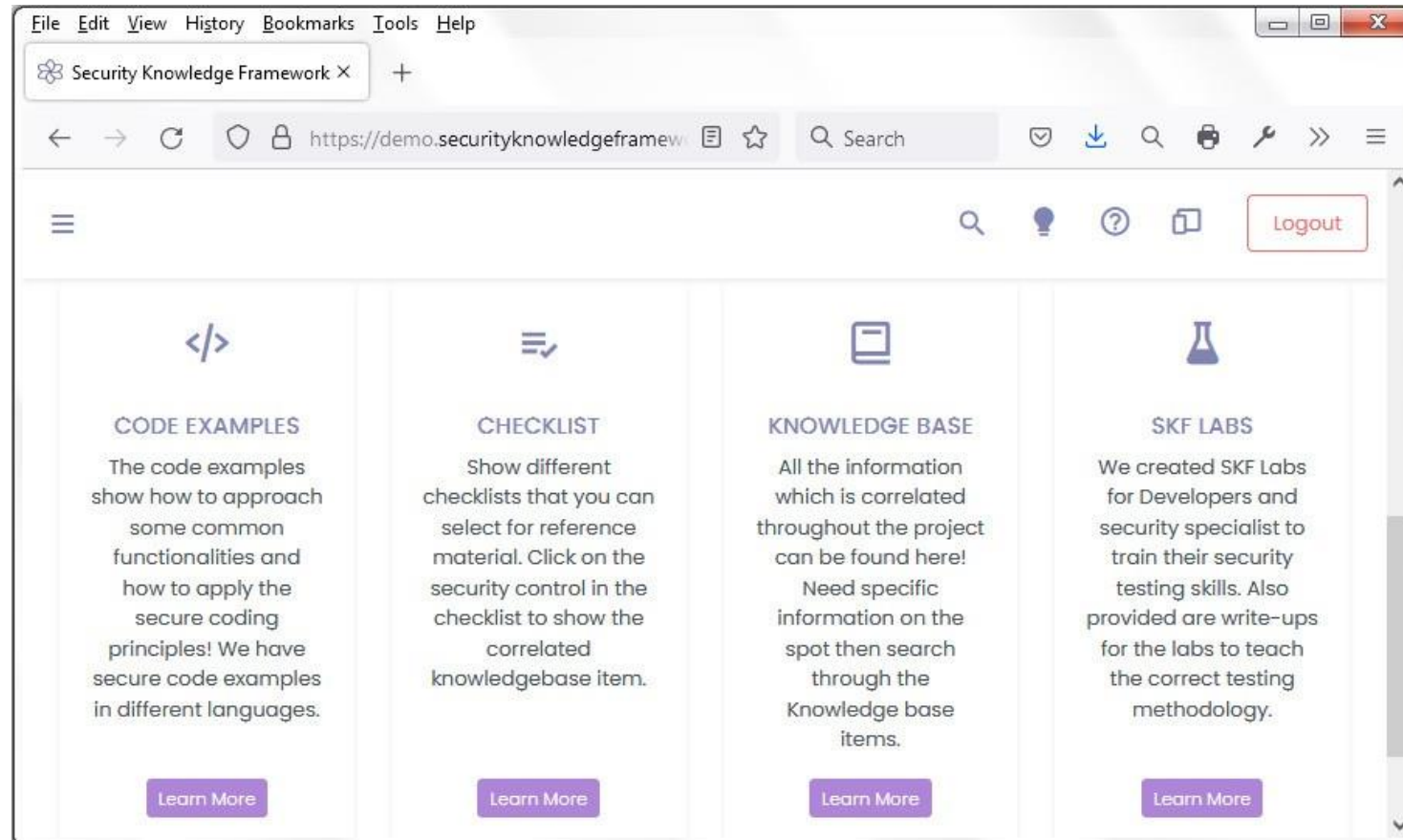
Bobby Tables

- Code examples to prevent **SQL injection** in **Bobby Tables**:
- <https://bobby-tables.com/python>



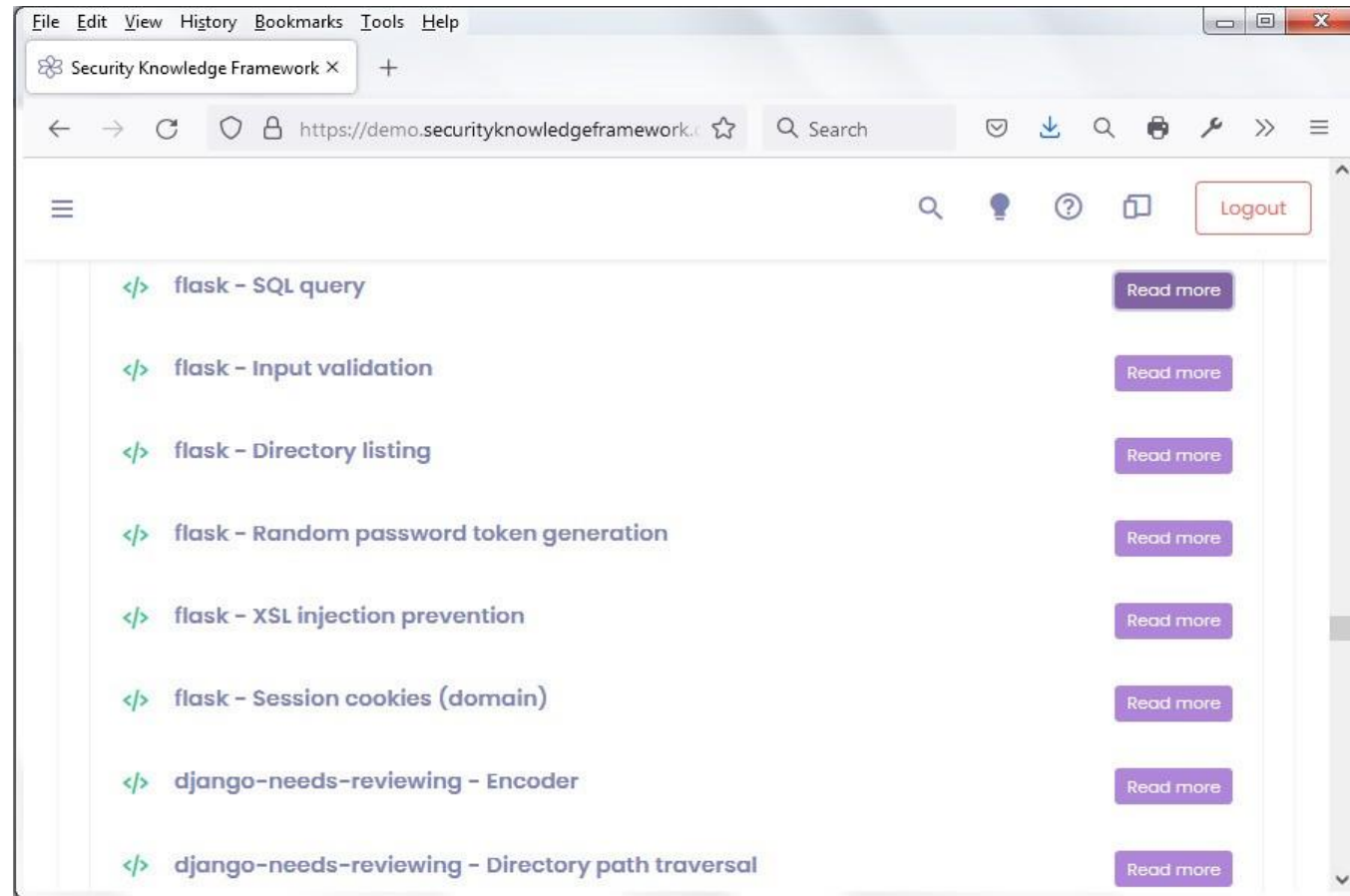
OWASP Security Knowledge Framework (SKF)

- It contains **best practice code examples** showing how to prevent hackers gaining access and running exploits in your application.



OWASP Security Knowledge Framework (SKF)

- Code examples for different languages and frameworks in [OWASP Security Knowledge Framework](#)



OWASP Secure Coding Practices Quick Reference Guide

It defines a set of **security coding practices** in a **checklist format** that can be adopted into building secure software.

- Input validation
- Output encoding
- Authentication and password management
- Session management
- Access control
- Cryptographic practices
- Error handling and logging
- Data protection
- Communication security
- **Database security**
- **File management**
- General coding practices



November 2010

OWASP Secure Coding Practices Quick Reference Guide

Copyright and License

Copyright © 2010 The OWASP Foundation.

This document is released under the Creative Commons Attribution ShareAlike 3.0 license. For any reuse or distribution, you must make clear to others the license terms of this work.
<http://creativecommons.org/licenses/by-sa/3.0/>

Version 2.0

1

OWASP Secure Coding Practices Quick Reference Guide

Database Security:

- ☐ Use strongly typed [*parameterized queries*](#)
- ☐ Utilize input validation and output encoding and be sure to address meta characters. If these fail, do not run the database command
- ☐ Ensure that variables are strongly typed
- ☐ The application should use the lowest possible level of privilege when accessing the database
- ☐ Use secure credentials for database access
- ☐ Connection strings should not be hard coded within the application. Connection strings should be stored in a separate configuration file on a trusted system and they should be encrypted.
- ☐ Use stored procedures to abstract data access and allow for the removal of permissions to the base tables in the database
- ☐ Close the connection as soon as possible
- ☐ Remove or change all default database administrative passwords. Utilize strong passwords/phrases or implement multi-factor authentication
- ☐ Turn off all unnecessary database functionality (e.g., unnecessary stored procedures or services, utility packages, install only the minimum set of features and options required (surface area reduction))
- ☐ Remove unnecessary default vendor content (e.g., sample schemas)
- ☐ Disable any default accounts that are not required to support business requirements
- ☐ The application should connect to the database with different credentials for every trust distinction (e.g., user, read-only user, guest, administrators)

OWASP Secure Coding Practices Quick Reference Guide

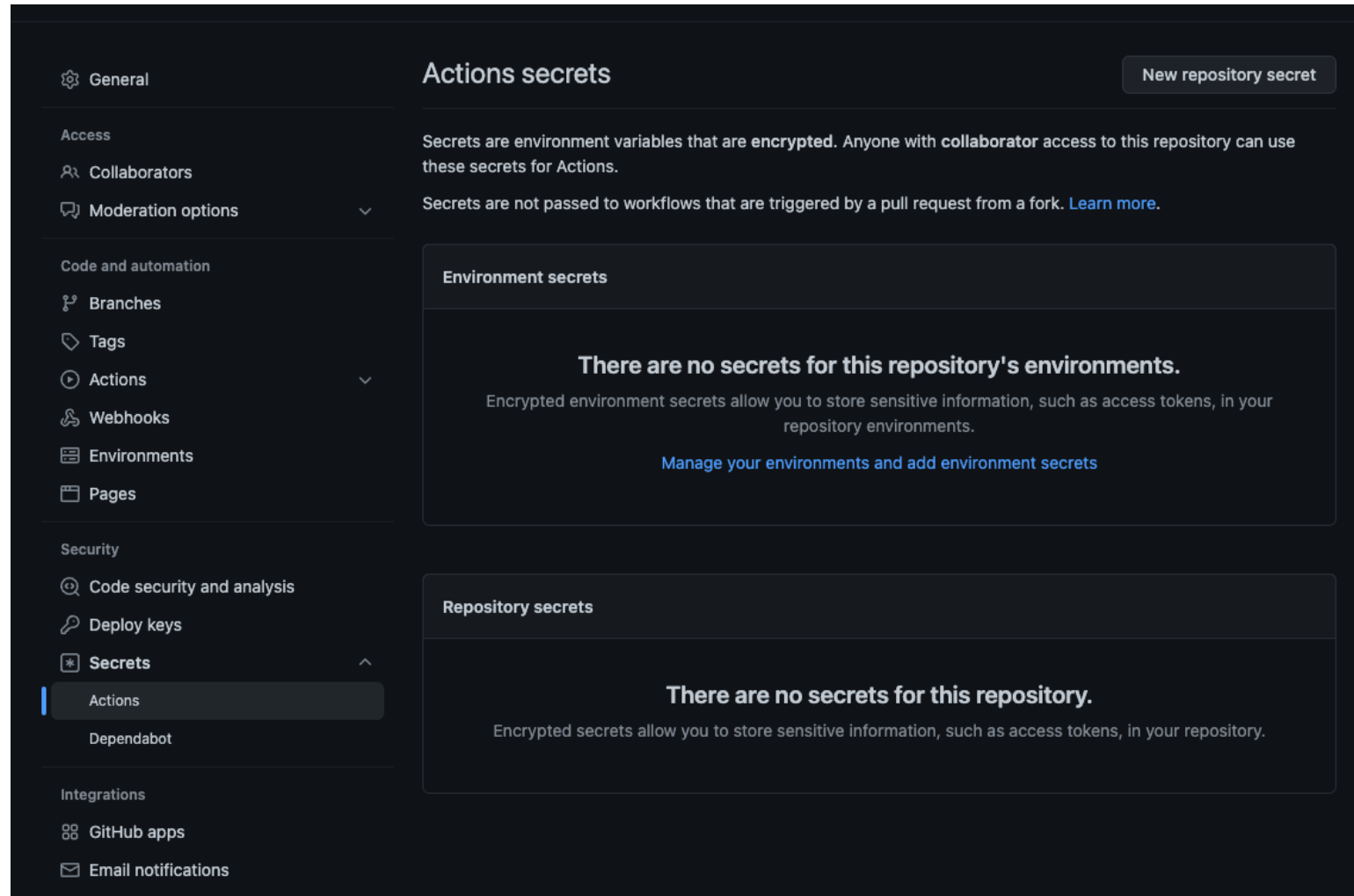
File Management:

- ☐ Do not pass user supplied data directly to any dynamic include function
- ☐ Require authentication before allowing a file to be uploaded
- ☐ Limit the type of files that can be uploaded to only those types that are needed for business purposes
- ☐ Validate uploaded files are the expected type by checking file headers. Checking for file type by extension alone is not sufficient
- ☐ Do not save files in the same web context as the application. Files should either go to the content server or in the database.
- ☐ Prevent or restrict the uploading of any file that may be interpreted by the web server.
- ☐ Turn off execution privileges on file upload directories
- ☐ Implement safe uploading in UNIX by mounting the targeted file directory as a logical drive using the associated path or the chrooted environment
- ☐ When referencing existing files, use a white list of allowed file names and types. Validate the value of the parameter being passed and if it does not match one of the expected values, either reject it or use a hard coded default file value for the content instead
- ☐ Do not pass user supplied data into a dynamic redirect. If this must be allowed, then the redirect should accept only validated, relative path URLs
- ☐ Do not pass directory or file paths, use index values mapped to pre-defined list of paths
- ☐ Never send the absolute file path to the client
- ☐ Ensure application files and resources are read-only
- ☐ Scan user uploaded files for viruses and malware

Question time

- Remember the question “how to secure your database credential?”

(Maybe) Solution



The screenshot shows the GitHub Actions secrets page for a repository. The left sidebar contains navigation links: General, Access (Collaborators, Moderation options), Code and automation (Branches, Tags, Actions, Webhooks, Environments, Pages), Security (Code security and analysis, Deploy keys, Secrets, Actions, Dependabot), and Integrations (GitHub apps, Email notifications). The 'Actions' link under Security is highlighted. The main content area is titled 'Actions secrets' and includes a 'New repository secret' button. It explains that secrets are encrypted environment variables and are not passed to workflows from forks. Below this, there are two sections: 'Environment secrets' and 'Repository secrets'. Both sections state that there are no secrets for this repository and provide a link to 'Manage your environments and add environment secrets'.

Actions secrets [New repository secret](#)

Secrets are environment variables that are **encrypted**. Anyone with **collaborator** access to this repository can use these secrets for Actions.

Secrets are not passed to workflows that are triggered by a pull request from a fork. [Learn more](#).

Environment secrets

There are no secrets for this repository's environments.

Encrypted environment secrets allow you to store sensitive information, such as access tokens, in your repository environments.

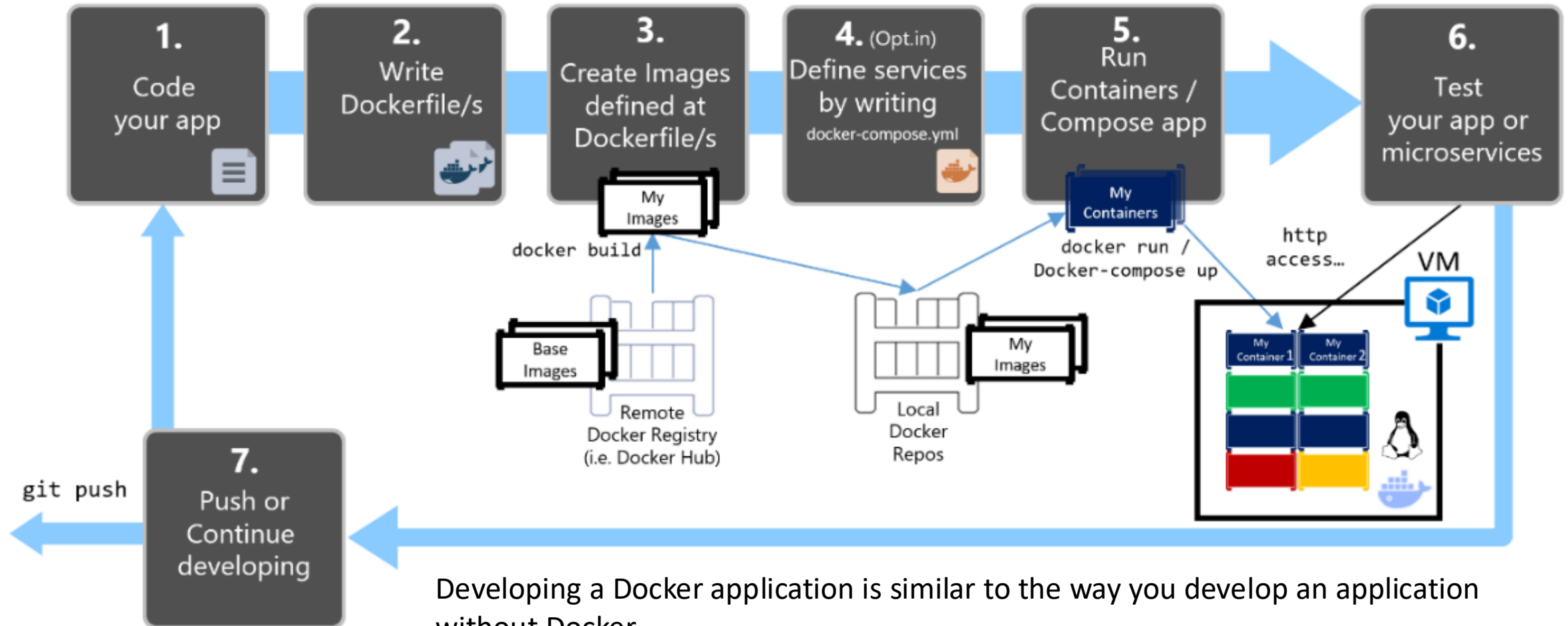
[Manage your environments and add environment secrets](#)

Repository secrets

There are no secrets for this repository.

Encrypted secrets allow you to store sensitive information, such as access tokens, in your repository.

Development workflow for Docker apps



Developing a Docker application is similar to the way you develop an application without Docker.

The difference is that while developing for Docker, you're deploying and testing your application or services running within Docker containers in your local environment.

Dockerfile

- Previous lab introduce the concept to execute docker image.
- It is possible to write the team's own docker image to simplify the development and deployment process.
- A Dockerfile is a text document that contains all the commands a user could call on the command line to assemble an image.

```
# syntax=docker/dockerfile:1
FROM node:12-alpine
RUN apk add --no-cache python2 g++ make
WORKDIR /app
COPY . .
RUN yarn install --production
CMD ["node", "src/index.js"]
EXPOSE 3000
```

docker-compose

- Compose is a tool for defining and running multi-container Docker applications.
- Compose use a YAML file to configure application's services.
- With a single command, it can create and start all the services from the configuration.

```
version: "3.9" # optional since v1.27.0
services:
  web:
    build: .
    ports:
      - "8000:5000"
    volumes:
      - ./code
      - logvolume01:/var/log
    depends_on:
      - redis
  redis:
    image: redis
volumes:
  logvolume01: {}
```

QUESTION AND COMMENTS?

Reference

- <https://healthitsecurity.com/news/proprietary-info-source-code-stolen-in-lastpass-password-manager-data-breach>
- <https://owasp.org/www-project-proactive-controls/v3/en/c9-security-logging>
- https://owasp.org/www-community/attacks/Regular_expression_Denial_of_Service_-_ReDoS
- <https://www.bleepingcomputer.com/news/security/microsoft-teams-stores-auth-tokens-as-plaintext-in-windows-linux-macs/>
- <https://learn.microsoft.com/en-us/dotnet/architecture/microservices/docker-application-development-process/docker-app-development-workflow>